Lastenheft: Kompass

Nov 6, 2026 · @Massimo

Kompass ist ein interner Wissens-Chatbot, der Fragen von Mitarbeitenden aus den Dokumenten der Firma beantwortet und dabei die Quellen nennt.

1. Ausgangslage & Zielsetzung

Mitarbeitende verlieren Zeit, weil Wissen in vielen Dokumenten verstreut ist; Kompass soll Fragen dazu in Sekunden mit Quellenangabe beantworten.

Ausgangslage: In der fiktiven Firma liegen Reglemente, Richtlinien und Preislisten als PDF, Word und Excel auf verschiedenen Laufwerken. Wer etwas sucht, fragt Kolleginnen oder sucht lange selbst.

Ziele:

- Fragen in natürlicher Sprache beantworten, nur auf Basis der eigenen Dokumente
- Jede Antwort nennt Dokument und Seite, damit man sie prüfen kann
- Mitarbeitende sehen nur Inhalte, für die sie berechtigt sind
- Die Antwortqualität ist messbar

Projektzweck: Kompass ist ein Portfolio-Projekt für die Bewerbung als Junior-Entwickler. Es soll saubere Softwareentwicklung zeigen: Datenbank, Login, Tests, CI und Docker.

2. Produkteinsatz

Kompass läuft bei der fiktiven **Muster Haustechnik AG**, einem Schweizer KMU mit rund 50 Mitarbeitenden in drei Abteilungen.

Abteilungen und typische Dokumente:

Abteilung	Beispiel-Dokumente
HR	Personalreglement, Spesenreglement, Ferien- und Absenzenregelung
IT	IT-Richtlinie, Passwort-Weisung, Anleitung VPN
Verkauf	Preisliste (Excel), Lieferbedingungen, Produktdatenblätter
Alle	Firmenleitbild, Notfallkonzept, Organigramm

Benutzerrollen:

Rolle	Darf
Mitarbeitende (USER)	Fragen stellen, Antworten aus Dokumenten der eigenen Abteilung und aus "Alle" erhalten
Administrator (ADMIN)	Zusätzlich Dokumente hochladen, einer Abteilung zuordnen, löschen; Benutzer verwalten

Beispiel-Szenarien:

1. Ein Monteur fragt: "Wie viel Spesen bekomme ich für ein Mittagessen auswärts?" Kompass antwortet mit dem Betrag und nennt Spesenreglement, Seite 2.
2. Eine Verkäuferin fragt nach dem Preis eines Produkts. Kompass liest die Antwort aus der Excel-Preisliste.
3. Ein IT-Mitarbeiter fragt nach dem Lohnreglement. Kompass antwortet, dass dazu keine Information verfügbar ist, weil das Dokument nur für HR freigegeben ist.
4. Die HR-Leiterin lädt ein neues Reglement hoch. Es ist nach weniger als einer Minute abfragbar.

3. Funktionale Anforderungen

Elf Muss-Anforderungen bilden das MVP; Soll- und Kann-Anforderungen folgen nur, wenn das MVP fertig und getestet ist.

ID	Anforderung	Priorität
F01	Admin lädt Dokumente als PDF, DOCX oder XLSX hoch (max. 20 MB pro Datei)	Muss
F02	Admin ordnet jedes Dokument einer Abteilung zu: HR, IT, Verkauf oder Alle	Muss
F03	Das System liest den Text aus, teilt ihn in Abschnitte und speichert sie als Vektoren	Muss
F04	Dokumentenliste mit Name, Abteilung, Upload-Datum und Status (in Verarbeitung, bereit, Fehler)	Muss
F05	Admin löscht ein Dokument; alle zugehörigen Abschnitte werden mitgelöscht	Muss
F06	Benutzer stellt eine Frage im Chat und erhält eine Antwort auf Deutsch	Muss

ID	Anforderung	Priorität
F07	Antworten stützen sich nur auf gefundene Abschnitte; ohne Treffer kommt der Hinweis "Dazu habe ich keine Information"	Muss
F08	Unter jeder Antwort stehen die Quellen: Dokumentname und Seite bzw. Tabellenblatt	Muss
F09	Login mit Benutzername und Passwort, Rollen USER und ADMIN	Muss
F10	Die Suche liefert nur Abschnitte aus der eigenen Abteilung und aus "Alle"	Muss
F11	Der LLM-Anbieter ist per Konfiguration umschaltbar: Ollama (Standard), Claude oder OpenAI	Muss
F12	Excel-Zeilen werden mit ihren Spaltennamen gespeichert, z. B. "Produkt: X, Preis: 120 CHF"	Soll
F13	Folgefragen innerhalb einer Sitzung berücksichtigen den bisherigen Chat	Soll
F14	Ein Testfragen-Set läuft automatisiert und gibt eine Trefferquote aus	Soll
F15	Benutzerverwaltung in der Oberfläche (anlegen, Abteilung zuweisen)	Kann
F16	Feedback pro Antwort (Daumen hoch/runter), gespeichert in der DB	Kann
F17	Antworten erscheinen schrittweise (Streaming)	Kann
F18	Chat-Verlauf bleibt über Sitzungen hinweg gespeichert	Kann

Für das MVP reichen Demo-Benutzer, die beim Start automatisch angelegt werden (F15 entfällt damit vorerst).

4. Nicht-funktionale Anforderungen

Die Zielwerte sind bewusst einfach messbar, damit du sie im README belegen kannst.

ID	Bereich	Anforderung	Zielwert
NF01	Antwortzeit	Zeit bis zur vollständigen Antwort	Lokal (Ollama, Laptop) ⩽ 30 s, Cloud-API ⩽ 10 s
NF02	Verarbeitung	Hochgeladenes Dokument ist abfragbar	PDF mit 10 Seiten in ⩽ 60 s
NF03	Antwortqualität	Richtiges Dokument unter den Quellen	⩾ 80 % der 20 Testfragen
NF04	Datenschutz	Im lokalen Modus verlassen keine Daten den Rechner	Kein externer Netzwerkaufruf im Ollama-Modus
NF05	Sicherheit	Passwörter gehasht, API-Keys nur als Umgebungsvariablen	BCrypt; kein Key im Repo
NF06	Bedienbarkeit	Oberfläche auf Deutsch, nutzbar auf Laptop und Handy	Keine horizontale Scrollleiste ab 375 px Breite
NF07	Wartbarkeit	Code, Kommentare und README auf Englisch; einheitlicher Stil	ktlint ohne Fehler
NF08	Testbarkeit	Automatische Tests laufen bei jedem Push	CI grün; Kernlogik mit Unit- und Integrationstests
NF09	Installierbarkeit	Ein Fremder startet das Projekt	`docker compose up`, lauffähig in ⩽ 5 Minuten

5. Rahmenbedingungen

Kompass läuft komplett lokal per Docker; Recruiter sehen es über ein Demo-Video im README.

- **Technik:** Kotlin, Spring Boot 4, Spring AI 2.0, PostgreSQL mit pgvector, Thymeleaf + HTMX, Gradle (Kotlin DSL)
- **Sprachmodell:** Ollama lokal als Standard; Claude- oder OpenAI-API per Konfiguration umschaltbar (F11)
- **Embeddings:** Ollama-Modell `nomic-embed-text` (Claude bietet keine eigenen Embeddings an)
- **Sprache:** Oberfläche Deutsch; Code, Kommentare, Commits und README Englisch
- **Betrieb:** nur lokal mit `docker compose up`; keine öffentliche Online-Demo

- **Präsentation:** Demo-Video (1–2 Minuten) und Screenshots im README
- **Daten:** ausschliesslich selbst erstellte, fiktive Dokumente der Muster Haustechnik AG; keine echten Firmendaten
- **Lizenz:** MIT, Repository öffentlich auf GitHub
- **Aufwand:** Einzelprojekt, rund 10 Stunden pro Woche über etwa 8 Wochen

6. Abgrenzung

Folgendes gehört bewusst nicht zum Projekt, damit der Umfang machbar bleibt:

- Keine Anbindung an SharePoint, Google Drive oder Netzlaufwerke; Dokumente kommen nur per Upload
- Keine Texterkennung (OCR) für gescannte PDFs
- Keine Bilder, Grafiken oder Diagramme in Dokumenten auswerten
- Kein Single Sign-on (z. B. Microsoft-Login)
- Keine Agenten, die selbstständig Aktionen ausführen
- Kein Training oder Fine-Tuning eigener Modelle
- Keine Mehrmandantenfähigkeit (nur eine Firma)
- Kein Hosting im Internet und kein Betrieb für echte Kunden

7. Abnahmekriterien

Das MVP gilt als fertig, wenn alle folgenden Punkte erfüllt sind:

- [ ] Alle Muss-Anforderungen (F01–F11) sind umgesetzt
- [ ] Die vier Szenarien aus Kapitel 2 funktionieren wie beschrieben
- [ ] Szenario 3 ist als automatisierter Test abgedeckt: Ein IT-Benutzer erhält keine HR-Inhalte
- [ ] Mindestens 80 % der 20 Testfragen liefern das richtige Dokument als Quelle (NF03)
- [ ] Umschalten zwischen Ollama und einer Cloud-API funktioniert nur über Konfiguration, ohne Codeänderung
- [ ] CI ist grün, ktlint meldet keine Fehler
- [ ] `docker compose up` startet App, Datenbank und Beispieldaten auf einem frischen Rechner
- [ ] README enthält Demo-Video, Schnellstart, Architektur-Skizze und Evaluations-Ergebnis

8. Offene Punkte

Diese Fragen müssen vor oder während der Umsetzung geklärt werden; keine davon blockiert den Start.

- [ ] **Hardware:** Wie viel RAM und welche Grafikkarte hat dein Rechner? Das bestimmt, welches Ollama-Modell flüssig läuft, und ob NF01 realistisch ist.
- [ ] **Ollama nativ oder in Docker?** Auf dem Mac nutzt Ollama die GPU nur nativ, nicht im Container.
- [ ] **Cloud-Anbieter:** Mit welcher API testest du den Cloud-Modus zuerst, Claude oder OpenAI?
- [ ] **Testdokumente:** Wie viele genau, und schreibst du sie selbst oder mit KI-Hilfe? Vorschlag: 12 Dokumente, 4 pro Abteilung.
- [ ] **Chunk-Grösse:** Startwert festlegen und in der Evaluation vergleichen.
- [ ] **Demo-Benutzer:** Namen und Abteilungen der Seed-Benutzer festlegen, z. B. ein Benutzer pro Abteilung plus ein Admin.